DOCUMENTAÇÃO TÉCNICA DO PROJETO ECO MOBILITY - SOULUP

Disciplina: Domain Driven Design Using Java
Projeto: Eco Mobility
Instituição: Faculdade de Informática e Administração Paulista
Equipe: SoulDevs

Integrantes:
Arthur Arruda da Silva - RM 569593
Enzo Torres Estanguera - RM 568937
Giovane Dias Chicoli - RM 569750
Álvaro Morais Holanda - RM 568607
Dhiego Gozzo Belmonte - RM 571505

SUMÁRIO

1. OBJETIVO E ESCOPO DO PROJETO
2. DESCRIÇÃO DAS FUNCIONALIDADES
3. ENDPOINTS DA API REST
4. PROTÓTIPO DO SISTEMA
5. MODELO DE ENTIDADE-RELACIONAMENTO (MER)
6. DIAGRAMA DE CLASSES E ARQUITETURA DO SISTEMA
7. PROCEDIMENTOS PARA EXECUÇÃO DA APLICAÇÃO

1. OBJETIVO E ESCOPO DO PROJETO

1.1 Propósito Principal

O Eco Mobility SoulUp é uma solução desenvolvida em Java com o objetivo de incentivar hábitos sustentáveis relacionados à mobilidade urbana. A aplicação utiliza conceitos de gamificação para estimular os usuários a realizar missões ecológicas, acumular pontos e trocá-los por vouchers.

O sistema disponibiliza uma API RESTful desenvolvida com Spring Boot, responsável por expor os recursos da aplicação e permitir a comunicação com clientes externos. A persistência dos dados é realizada por meio de conexão JDBC com o banco de dados Oracle.

A solução também contempla missões sustentáveis com informações sobre estimativas de redução de CO_2, além de mecanismos de pontuação e recompensas que incentivam a participação dos usuários em práticas de mobilidade mais sustentáveis.

1.2 Escopo do Sistema

As principais atividades contempladas pelo sistema são:

- **Cadastro, consulta, atualização e exclusão de usuários, conforme os endpoints implementados.**
- **Autenticação de usuários por e-mail e senha.**
- **Consulta, cadastro, atualização e exclusão de missões sustentáveis.**
- **Registro da conclusão de missões, respeitando as regras de negócio implementadas.**
- **Atribuição de pontos aos usuários pelas missões concluídas.**
- **Consulta, cadastro, atualização e exclusão de vouchers.**
- **Resgate de vouchers utilizando os pontos acumulados.**
- **Registro, consulta, atualização e exclusão do histórico de resgates.**
- **Consulta de informações relacionadas à pontuação dos usuários e às missões sustentáveis.**
- **Disponibilização dos recursos por meio de endpoints REST.**
- **Publicação e acesso à API por meio de um serviço de hospedagem em nuvem.**

Fora do escopo

Não fazem parte do escopo atual, salvo se forem implementados posteriormente:

- **Rastreamento de deslocamentos por GPS em tempo real.**
- **Monitoramento automático dos meios de transporte utilizados.**
- **Processamento de pagamentos em dinheiro.**
- **Integração com gateways de pagamento.**
- **Integração direta com aplicativos de transporte público.**
- **Desenvolvimento e integração de uma interface frontend com a API nesta etapa do**

projeto.

Observação: A API foi desenvolvida para permitir o consumo de seus recursos por aplicações externas, como uma interface web. Entretanto, a versão atual da entrega contempla o backend, sem uma interface frontend integrada.

2. DESCRIÇÃO DAS FUNCIONALIDADES

2.1 Gestão de Usuários

A API disponibiliza recursos para operações relacionadas aos usuários, incluindo cadastro, consulta, atualização, exclusão e autenticação.

Os dados de usuário incluem nome, e-mail, senha e saldo de pontos. A senha é recebida durante o cadastro e a autenticação, mas não deve ser retornada nas respostas da API, garantindo que essa informação não seja exposta nas consultas.

2.2 Catálogo de Missões Sustentáveis

O sistema disponibiliza missões relacionadas a práticas sustentáveis de mobilidade. Cada missão pode conter nome, descrição, pontuação, tipo de transporte, estimativa de CO_2, quantidade de conclusões, indicação de missão diária e data de conclusão.

A API permite cadastrar, consultar, atualizar e excluir missões por meio dos endpoints correspondentes.

2.3 Conclusão de Missões

A aplicação disponibiliza uma operação para registrar a conclusão de uma missão associada a um usuário.

Quando uma missão é concluída com sucesso, o sistema registra a conclusão, adiciona os pontos correspondentes ao saldo do usuário e atualiza a quantidade de conclusões da missão.

Para missões diárias, o sistema verifica se o usuário já realizou a mesma missão na data atual, evitando conclusões duplicadas no mesmo dia.

2.4 Consulta de Pontos

Os pontos representam as recompensas acumuladas pelo usuário ao concluir missões. O saldo pode ser consultado por meio dos recursos de usuário e utilizado nas operações de resgate de vouchers.

O sistema também realiza atualizações do saldo durante as operações de conclusão de missões e resgate de vouchers, respeitando as regras de negócio implementadas.

2.5 Resgate de Vouchers

A API disponibiliza recursos para cadastrar, consultar, atualizar e excluir vouchers, que possuem código, descrição, custo em pontos e valor correspondente.

O resgate utiliza os pontos acumulados pelo usuário. Antes de realizar a operação, o sistema verifica se o saldo é suficiente. Quando o resgate é concluído, os pontos correspondentes são descontados e a operação é registrada no histórico.

2.6 Histórico de Resgates

O sistema permite registrar, consultar, atualizar e excluir registros do histórico de resgates, incluindo o código do voucher, os pontos utilizados e a data da operação, conforme os dados persistidos no banco Oracle.

Os registros são associados aos usuários, permitindo consultar o histórico correspondente a cada e-mail informado na requisição.

2.7 Trajetória Ecológica

A solução contempla informações associadas à pontuação dos usuários, às missões concluídas e às estimativas de redução de CO_2 relacionadas às missões.

Essas informações são disponibilizadas por meio dos recursos da API. Sua apresentação em uma interface visual depende de uma aplicação cliente que consuma os endpoints, não fazendo parte da integração implementada nesta entrega.

3. TABELA DE ENDPOINTS DA API RESTful

A API do Eco Mobility SoulUp foi desenvolvida com Spring Boot e disponibilizada na nuvem por meio da plataforma Render.

URL base: https://soulup-api-78mu.onrender.com

A tabela apresenta os principais recursos REST implementados na aplicação e seus respectivos métodos HTTP, URIs e códigos de resposta de sucesso.

Recurso	Método HTTP	URI	Resposta de sucesso
Verificar status da API	GET	`/api/status`	200 OK
Listar usuários	GET	`/api/usuarios`	200 OK
Consultar usuário por e-mail	GET	`/api/usuarios/{email}`	200 OK
Cadastrar usuário	POST	`/api/usuarios`	201 Created
Atualizar usuário	PUT	`/api/usuarios/{email}`	200 OK
Excluir usuário	DELETE	`/api/usuarios/{email}`	204 No Content
Realizar login	POST	`/api/usuarios/login`	200 OK
Listar missões	GET	`/api/missoes`	200 OK
Consultar missão por nome	GET	`/api/missoes/{nome}`	200 OK
Cadastrar missão	POST	`/api/missoes`	201 Created

Atualizar missão	PUT	`/api/missoes/{nome}`	200 OK
Excluir missão	DELETE	`/api/missoes/{nome}`	204 No Content
Registrar conclusão de missão	POST	`/api/missoes/{nome}/conclusoes?email={email}`	**201 Created**
Listar vouchers	GET	`/api/vouchers`	200 OK
Consultar voucher por código	GET	`/api/vouchers/{codigo}`	200 OK
Cadastrar voucher	POST	`/api/vouchers`	201 Created
Atualizar voucher	PUT	`/api/vouchers/{codigo}`	200 OK
Excluir voucher	DELETE	`/api/vouchers/{codigo}`	204 No Content
Resgatar voucher	POST	`/api/vouchers/{codigo}/resgates?email={email}`	**201 Created**
Consultar histórico por e-mail	GET	`/api/historicos?email={email}`	200 OK
Registrar histórico	POST	`/api/historicos?email={email}`	201 Created

Atualizar histórico por código	PUT	`/api/historicos/{codigo}?email={email}`	200 OK
Excluir histórico por código	DELETE	`/api/historicos/{codigo}?email={email}`	204 No Content

Os códigos de sucesso apresentados correspondem aos retornos definidos nas classes Resource. Dependendo da operação, também podem ocorrer respostas de erro, como:

- `400 Bad Request`: dados ou parâmetros inválidos.
- `401 Unauthorized`: credenciais inválidas.
- `404 Not Found`: recurso não encontrado.
- `405 Method Not Allowed`: método HTTP não suportado.
- `409 Conflict`: conflito de dados ou restrição de integridade.
- `415 Unsupported Media Type`: tipo de conteúdo não suportado.
- `500 Internal Server Error`: erro interno inesperado.

3.1 Acesso à API publicada

A API está hospedada na plataforma Render e pode ser acessada pelo endereço:

https://soulup-api-78mu.onrender.com

Os seguintes endereços podem ser utilizados para consultar recursos disponíveis:

- **Status:** https://soulup-api-78mu.onrender.com/api/status
- **Missões:** https://soulup-api-78mu.onrender.com/api/missoes
- **Vouchers:** https://soulup-api-78mu.onrender.com/api/vouchers
- **Usuários:** https://soulup-api-78mu.onrender.com/api/usuarios

As requisições que modificam dados devem ser realizadas por um cliente HTTP, respeitando os métodos, parâmetros e corpos definidos pela API.

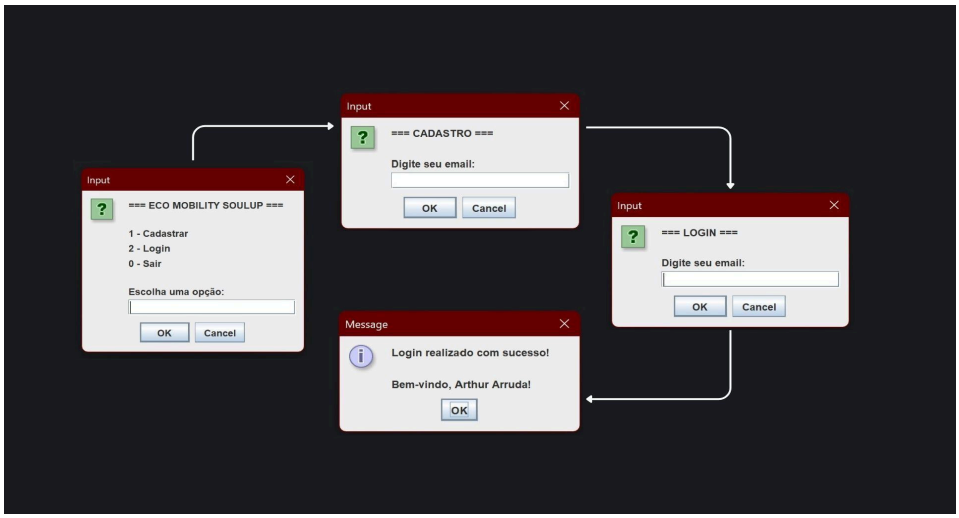
O endpoint de status também permite verificar a disponibilidade da conexão com o banco de dados Oracle por meio do recurso `/api/status/banco`.

4. PROTÓTIPO DO SISTEMA

O projeto Eco Mobility SoulUp possui registros visuais de um protótipo desenvolvido anteriormente em Java, utilizando caixas de diálogo por meio da classe `JOptionPane`. O protótipo foi utilizado para representar o fluxo de navegação e as principais funcionalidades propostas para a solução.

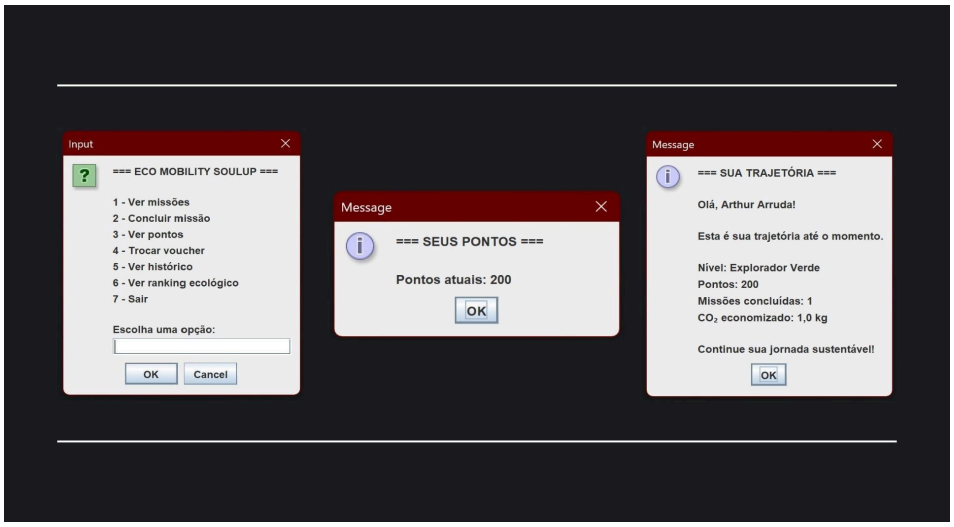
Na versão atual, a aplicação foi estruturada como uma API RESTful utilizando Java e Spring Boot, com persistência de dados em banco Oracle. Dessa forma, as imagens apresentadas nesta seção documentam o protótipo anterior e não representam uma interface gráfica integrada à API atualmente publicada.

Figura 1 – Cadastro e Login



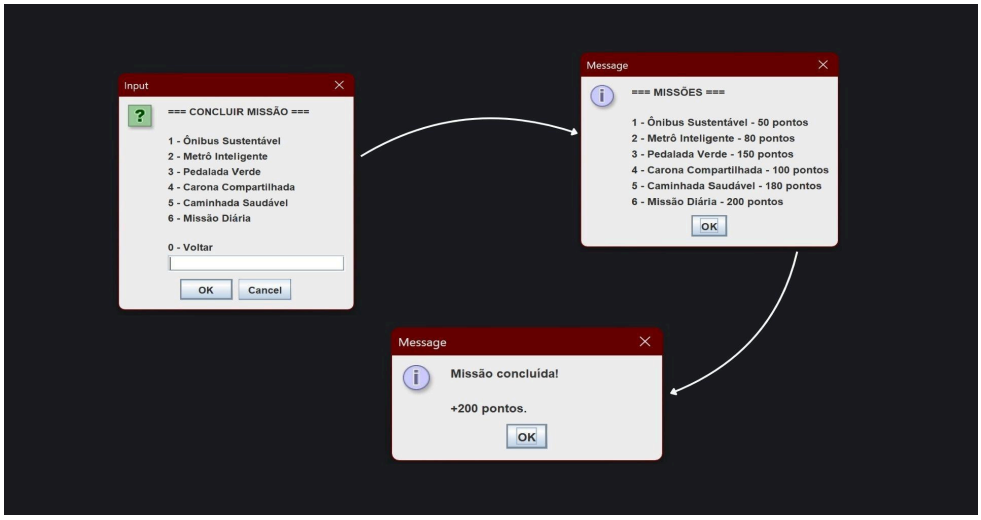
Descrição: o protótipo apresenta o menu inicial, com opções de cadastro e autenticação de usuários. Após a autenticação, o sistema informa o resultado da tentativa de acesso.

Figura 2 – Menu Principal, Pontuação e Trajetória Ecológica



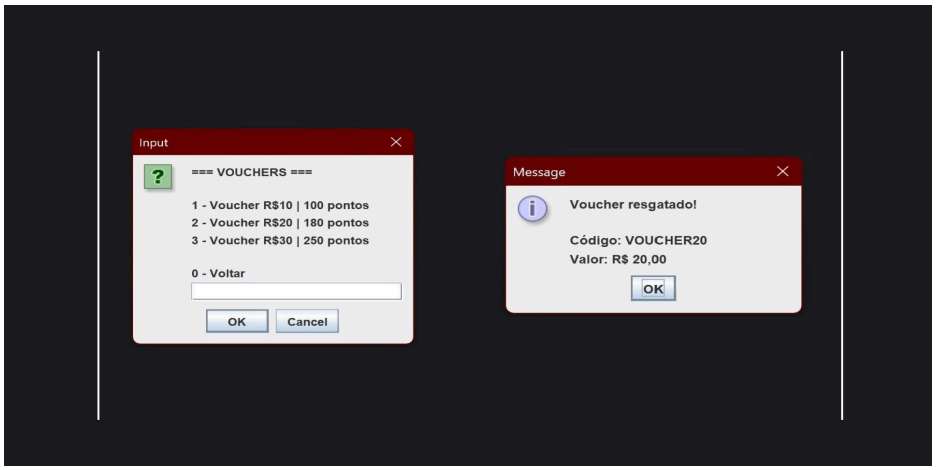
Descrição: o protótipo apresenta o menu principal e as opções para consultar a pontuação e a trajetória ecológica do usuário, incluindo seu nível de sustentabilidade, missões concluídas e estimativa de CO_2 economizado.

Figura 3 – Consulta e Conclusão de Missões



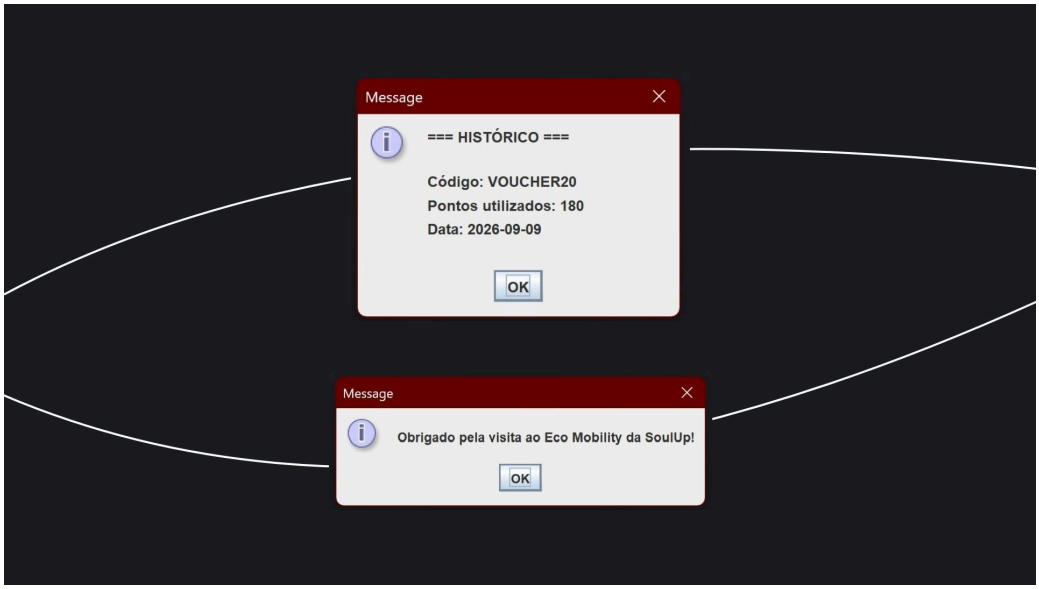
Descrição: o protótipo demonstra a consulta das missões sustentáveis e o fluxo de seleção e confirmação de uma missão, ilustrando a atribuição de pontos prevista pelas regras de negócio do sistema.

Figura 4 – Catálogo e Resgate de Vouchers



Descrição: o protótipo apresenta as opções de vouchers e seus respectivos custos em pontos, ilustrando o fluxo de seleção e confirmação de um resgate.

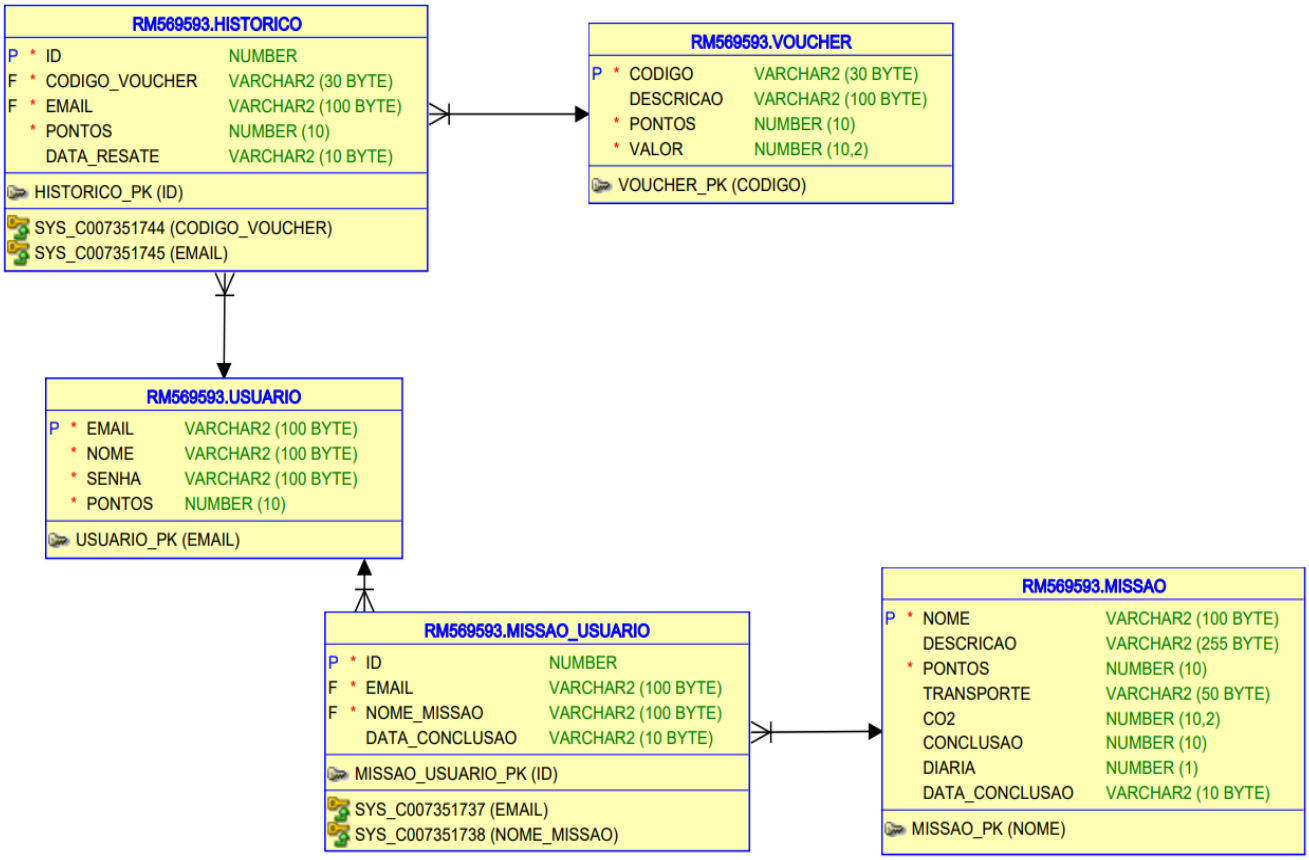
Figura 5 – Histórico de Resgates



Descrição: o protótipo apresenta o histórico de vouchers resgatados, com informações como código do voucher, pontos utilizados e data da operação.

5. MODELO DE ENTIDADE-RELACIONAMENTO (MER)

O Modelo de Entidade-Relacionamento (MER) representa a estrutura dos dados utilizados pelo sistema Eco Mobility SoulUp, demonstrando as principais entidades e seus relacionamentos no banco de dados Oracle.



O modelo apresenta as seguintes entidades e relacionamentos:

- **USUARIO → MISSAO_USUARIO: representa a associação entre os usuários e as missões concluídas.**
- **MISSAO → MISSAO_USUARIO: relaciona as missões aos respectivos registros de conclusão.**
- **USUARIO → HISTORICO: representa a associação dos registros de histórico aos usuários.**
- **VOUCHER → HISTORICO: relaciona os vouchers aos registros de resgate armazenados no histórico.**

A tabela `MISSAO_USUARIO` é utilizada para registrar as conclusões de missões realizadas pelos usuários. Já a tabela `HISTORICO` armazena informações relacionadas às operações de resgate de vouchers.

6. DIAGRAMA DE CLASSES E ARQUITETURA DO SISTEMA

Os diagramas apresentam a estrutura das classes implementadas na versão atual do SoulUp, desenvolvido em Java com Spring Boot. A aplicação está organizada em camadas, separando as responsabilidades de exposição dos endpoints REST, execução das regras de negócio, acesso aos dados e infraestrutura.

A camada Resource recebe as requisições HTTP e encaminha as operações às classes BO. A camada BO executa as regras de negócio, validações e operações do sistema. A camada DAO realiza a persistência e consulta dos dados no banco Oracle por meio da ConexaoFactory.

As entidades Usuario, Missao, Voucher e Historico representam os principais dados da aplicação. Os DTOs são utilizados para transportar informações em operações específicas, como autenticação, conclusão de missões e resgate de vouchers.

A infraestrutura inclui o CorsFilter, responsável pela configuração de acesso entre origens, e o GlobalExceptionHandler, que centraliza o tratamento de exceções. A classe SoulUpApplication inicializa a aplicação Spring Boot.

Os diagramas foram divididos em figuras para facilitar a leitura e apresentar os principais atributos, métodos e dependências das classes que compõem o projeto.

Figura 6.1 — Entidades e DTOs

Representa os dados manipulados pela aplicação e os objetos utilizados na comunicação da API.

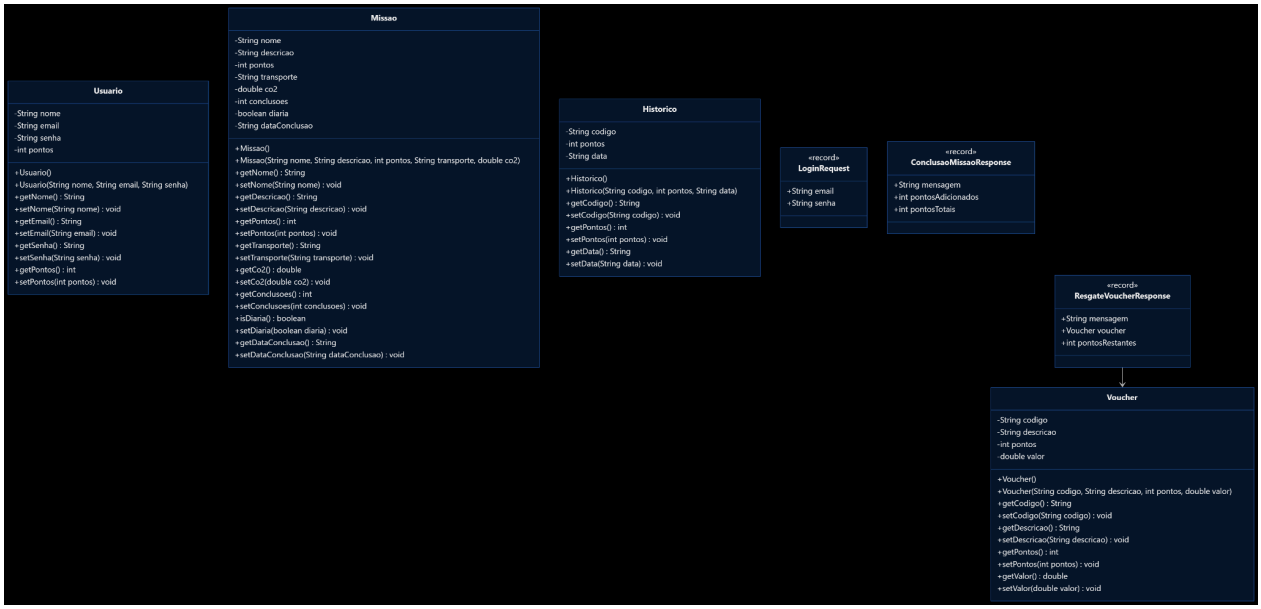


Figura 6.2 — Resources e endpoints

Representa as classes que recebem as requisições HTTP. Os nomes dos métodos correspondem aos métodos encontrados nos arquivos Resource.



Figura 6.3 — Camada de negócio (BO)

Apresenta as classes que executam as regras de negócio, validações, autenticação, conclusão de missões e resgate de vouchers.

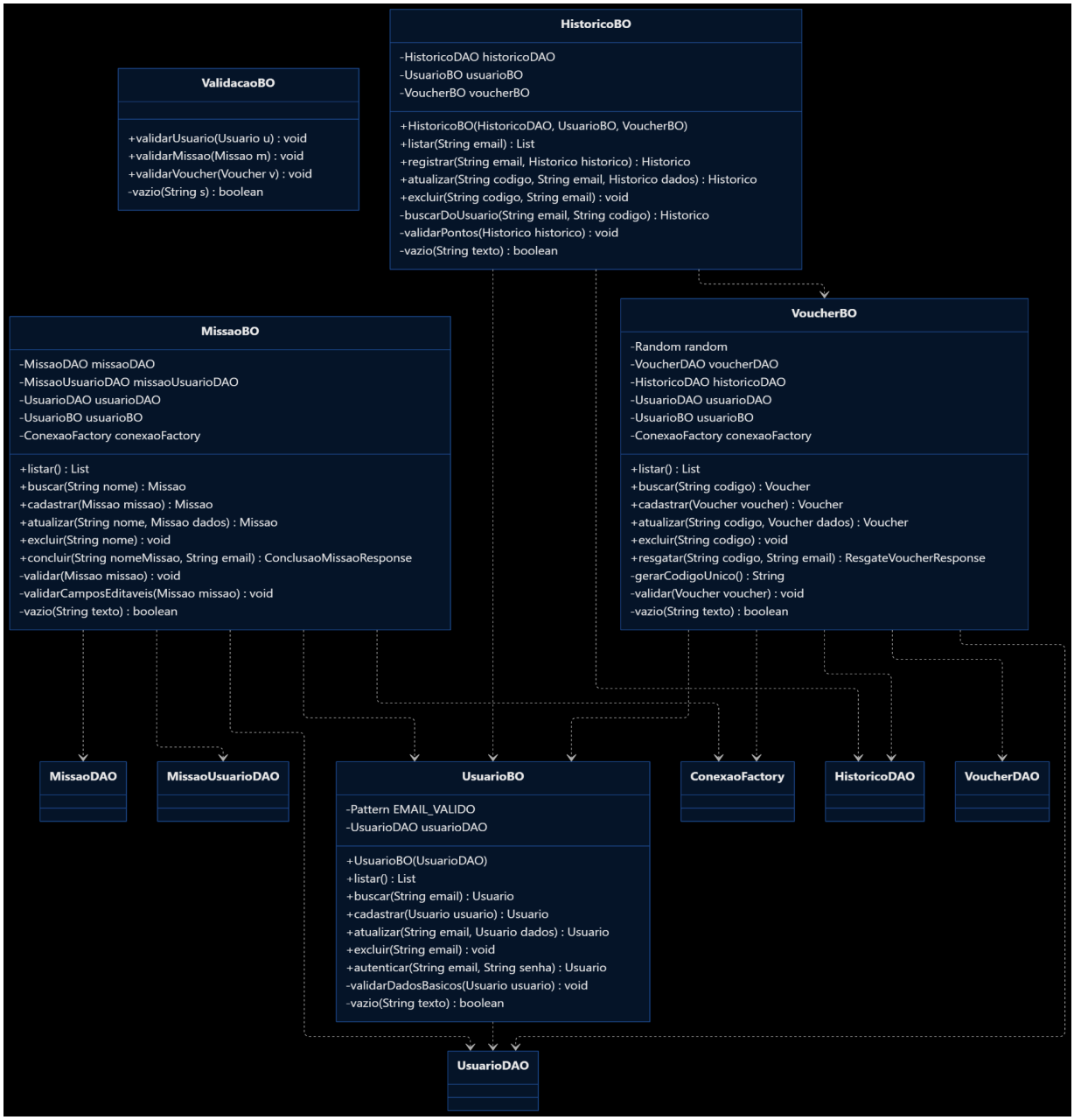


Figura 6.4 — Camada de acesso a dados (DAO)

Representa as classes que executam consultas, inserções, atualizações e exclusões no banco Oracle.



Figura 6.5 — Infraestrutura, exceções e inicialização



7. PROCEDIMENTOS PARA EXECUÇÃO DA APLICAÇÃO

7.1 Ferramentas e tecnologias utilizadas

- Java JDK 21;
- Spring Boot 3.4.5;
- IntelliJ IDEA;
- Apache Maven;
- Oracle Database;
- Oracle JDBC Driver;
- Oracle SQL Developer, para consulta e administração do banco de dados;
- Render, para hospedagem da API;
- Docker, para empacotamento e implantação da aplicação.

7.2 Configuração do ambiente

Para executar a aplicação localmente, é necessário ter o JDK 21 instalado e configurar o projeto no IntelliJ IDEA. O projeto utiliza Maven para gerenciar as dependências e realizar a compilação.

As configurações de conexão com o banco Oracle são obtidas por meio das seguintes variáveis de ambiente:

- `SOULUP_DB_URL`: endereço de conexão com o banco Oracle;
- `SOULUP_DB_USER`: usuário do banco de dados;
- `SOULUP_DB_PASSWORD`: senha do banco de dados.

Antes de executar a aplicação, essas variáveis devem estar configuradas corretamente no ambiente de execução.

A URL de conexão deve apontar para o banco Oracle utilizado pelo projeto. O usuário do banco precisa ter permissão para acessar as tabelas necessárias à aplicação.

A variável `SOULUP_DB_URL` possui uma URL padrão configurada na aplicação, correspondente ao banco Oracle da FIAP. Entretanto, as credenciais de acesso devem ser fornecidas por meio das variáveis de ambiente.

7.3 Execução da API

Para executar a aplicação localmente, siga os passos:

1. Abrir o projeto no IntelliJ IDEA.
2. Verificar se o projeto está configurado para utilizar o Java JDK 21.
3. Aguardar o carregamento das dependências Maven declaradas no arquivo `pom.xml`.
4. Configurar as variáveis de ambiente necessárias para a conexão com o banco Oracle.

5. Localizar a classe principal `br.com.fiap.SoulUpApplication`.
6. Executar a aplicação Spring Boot pelo IntelliJ IDEA.
7. Após a inicialização, acessar `http://localhost:8080/api/status` para verificar se a API está respondendo.
8. Acessar `http://localhost:8080/api/status/banco` para verificar a disponibilidade da conexão com o banco Oracle.

7.4 Acesso à aplicação publicada

A API está hospedada na plataforma Render e pode ser acessada pelo endereço:

https://soulup-api-78mu.onrender.com

O endpoint de status pode ser utilizado para verificar a disponibilidade do serviço:

https://soulup-api-78mu.onrender.com/api/status

Para verificar a conexão da aplicação publicada com o banco Oracle, também está disponível o endpoint:

https://soulup-api-78mu.onrender.com/api/status/banco

A API disponibiliza recursos REST para consulta e gerenciamento de usuários, missões, vouchers e histórico, conforme os endpoints implementados no projeto.

As operações que alteram dados devem ser realizadas por meio de requisições HTTP apropriadas, respeitando os métodos, parâmetros e corpos definidos pela API.